

MANIPAL INSTITUTE OF TECHNOLOGY
Manipal – 576104

SCHOOL OF COMPUTER ENGINEERING



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

CERTIFICATE

This is to certify that Ms./Mr. Reg. No.
..... Section: Roll No: has satisfactorily
completed the lab exercises prescribed for Parallel Computer Architecture and Programming Lab
[DSE 3126] of Fifth Year B. Tech. Degree at MIT, Manipal, in the academic year 2026-2027.

Date:

Signature
Faculty in Charge

Signature
Head of the Department

CONTENTS

LAB NO.	TITLE	PAGE NO.	REMARKS
	Course Objectives and Outcomes	iii	
	Evaluation plan	iii	
	Instructions to the Students	iv	
1	Introduction to execution environment of OpenMP	5	
2	OpenMP programs on work-sharing constructs	11	
3	OpenMP programs on synchronization constructs	13	
4	Introduction to execution environment of MPI	15	
5	Point to Point Communications in MPI	19	
6	Collective communications in MPI	23	
7	Programs on Arrays in CUDA	29	
8	Programs on Strings in CUDA	36	
9	Programs on Matrix in CUDA	41	
10	Programs on CUDA Device memory types and synchronization	44	
	References	48	

Course Objectives

- To understand, implement, optimize, and troubleshoot OpenMP parallelization in C/C++ programs, evaluating scalability and applying synchronization mechanisms effectively.
- Learn different APIs used in MPI for point to point, collective communications and error handling
- Learn how to write host and kernel code in CUDA for NVIDIA GPU card
- To develop the skills of design and implement parallel algorithms using different parallel programming environment

Course Outcomes

At the end of this course, students will be able to

- Describe the architecture and functionalities of modern processors, including multicore and manycore systems, and evaluate their performance using various metrics and laws of parallel programming.
- Develop shared-memory parallel programs, understanding key constructs like parallel regions, work-sharing, synchronization, and environment interaction to achieve performance improvements using OpenMP best practices.
- Skills in using MPI for parallel programming, including understanding MPI processes, messaging, data types, error handling, and process-to-process communication.
- Develop competence in CUDA programming, including understanding the CUDA programming model, memory hierarchy, and data-parallel execution model. They will be able to design and implement efficient parallel algorithms using CUDA.
- Optimize the performance of parallel programs by understanding and addressing memory access.

Evaluation plan

- Internal Assessment Marks : 60%

➤ Continuous Evaluation : 60%

Continuous evaluation component (for each evaluation):10 marks

The assessment will depend on punctuality, program execution, maintaining the observation note and answering the questions in viva voce.

- End semester assessment of 2 hour duration: 40 %

INSTRUCTIONS TO THE STUDENTS

Pre- Lab Session Instructions

1. Be in time and follow the institution dress code.
2. Must Sign in the log register provided.
3. Make sure to occupy the allotted seat and answer the attendance
4. Adhere to the rules and maintain the decorum.
5. Students must come prepared for the lab in advance.

In- Lab Session Instructions

- Follow the instructions on the allotted exercises.
- Show the program and results to the instructors on completion of experiments.
- On receiving approval from the instructor, copy the program and results in the Lab record
- Prescribed textbooks and class notes can be kept ready for reference if required.

General Instructions for the exercises in Lab

- Implement the given exercise individually and not in a group.
- Observation book should be complete with program, proper input output clearly showing the parallel execution in each process. Plagiarism (copying from others) is strictly prohibited and would invite severe penalty in evaluation.
- The exercises for each week are divided under three sets:
 - Solved example
 - Lab exercises - to be completed during lab hours
 - Additional Exercises - to be completed outside the lab or in the lab to enhance the skill
- In case a student misses a lab class, he/ she must ensure that the experiment is completed during the repetition class with the permission of the faculty concerned but credit will be given only to one day's experiment(s).
- Questions for lab tests and examination are not necessarily limited to the questions in the manual, but may involve some variations and / or combinations of the questions.

THE STUDENTS SHOULD NOT

- Bring mobile phones or any other electronic gadgets to the lab.
- Go out of the lab without permission.

Introduction to execution environment of OpenMP

Objectives:

In this lab, student will be able to

1. Understand the execution environment of OpenMP programs
2. Learn the various concept of OpenMP programming
3. Learn and use the compiler directives and library functions available in OpenMP

I.Introduction

Of many different parallel and distributed systems, multi-core and shared memory multiprocessors are most likely the easiest to program if only the right approach is taken.

Most modern CPUs are multi-core processors and, therefore, consist of a number of independent processing units called cores. Moreover, these CPUs support (simultaneous) multithreading(SMT)so that each core can(almost)simultaneously execute multiple independent streams of instructions called threads.

OpenMP, a parallel programming environment best suitable for writing parallel programs that are to be run on shared memory systems. It is not yet another programming language but an add-on to an existing language, usually Fortran or C/C++. The application programming interface (API) of OpenMP is a collection of

- compiler directives,
- supporting functions, and
- shell variables.

OpenMP compiler directives tell the compiler about the parallelism in the source code and provide instructions for generating the parallel code, i.e., the multi threaded translation of the source code. In C/C++, directives are always expressed as #pragmas. Supporting functions enable programmers to exploit and control the parallelism during the execution of a program. Shell variables permit tuning of compiled programs to a particular parallel system.

Compiling and Running an OpenMP Program

To illustrate different kinds of OpenMP API elements, we will start with a simple program

// OpenMP program to print Hello World using C language

```
#include <omp.h>      // OpenMP header
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main(int argc, char* argv[])
```

```
{
```

```

#pragma omp parallel      // Beginning of parallel region
{
    printf("Hello World... from thread = %d\n",
        omp_get_thread_num());
} // Ending of parallel region
}

```

Output:

```

sandhya@DESKTOP-3ERBGCG:~/HPC2024/OpenMP$ gcc -fopenmp Hello.c
sandhya@DESKTOP-3ERBGCG:~/HPC2024/OpenMP$ ./a.out
Hello World... from thread = 0
sandhya@DESKTOP-3ERBGCG:~/HPC2024/OpenMP$ export OMP_NUM_THREADS=5
sandhya@DESKTOP-3ERBGCG:~/HPC2024/OpenMP$ ./a.out
Hello World... from thread = 0
Hello World... from thread = 1
Hello World... from thread = 2
Hello World... from thread = 4
Hello World... from thread = 3
sandhya@DESKTOP-3ERBGCG:~/HPC2024/OpenMP$ _

```

When run for multiple time: Order of execution of threads changes every time.

```

sandhya@DESKTOP-3ERBGCG:~/HPC2024/OpenMP$ ./a.out
Hello World... from thread = 3
Hello World... from thread = 2
Hello World... from thread = 4
Hello World... from thread = 1
Hello World... from thread = 0
sandhya@DESKTOP-3ERBGCG:~/HPC2024/OpenMP$ ./a.out
Hello World... from thread = 0
Hello World... from thread = 1
Hello World... from thread = 3
Hello World... from thread = 2
Hello World... from thread = 4
sandhya@DESKTOP-3ERBGCG:~/HPC2024/OpenMP$

```

```

sandhya@DESKTOP-3ERBGCG:~/HPC2024/OpenMP$ time ./a.out
Hello World... from thread = 5
Hello World... from thread = 9
Hello World... from thread = 0
Hello World... from thread = 1
Hello World... from thread = 3
Hello World... from thread = 6
Hello World... from thread = 7
Hello World... from thread = 4
Hello World... from thread = 8
Hello World... from thread = 2

real    0m0.041s
user    0m0.000s
sys     0m0.016s

```

OpenMP: controlling the number of threads

Once a program is compiled, the number of threads can be controlled using the following shell variables:

- `OMP_NUM_THREADS` *comma-separated-list-of-positive-integers*
- `OMP_THREAD_LIMIT` *positive-integer*

The first one sets the number of threads the program should use (or how many threads should be used at every nested level of parallel execution). The second one limits the number of threads a program can use (and takes the precedence over `OMP_NUM_THREADS`).

Within a program, the following functions can be used to control the number of threads:

- `void omp_set_num_threads()` sets the number of threads used in the subsequent parallel regions without explicit specification of the number of threads;
- `int omp_get_num_threads()` returns the number of threads in the current team relating to the innermost enclosing parallel region;
- `int omp_get_max_threads()` returns the maximal number of threads available to the subsequent parallel regions;
- `int omp_get_thread_num()` returns the thread number of the calling thread within the current team of threads.

Parallelizing Loops with Independent Iterations

Printing out all integers from 1 to max in no particular order.

```
// Printing out all integers from 1 to max in no particular order.
#include <stdio.h>

#include <omp.h>

int main (int argc, char *argv[]) {

    int max; // sscanf (argv[1], "%d", &max);
    printf("\n Enter value of max\n");
    scanf("%d",&max);
    #pragma omp parallel for

    for (int i = 1; i <= max; i++)

        printf ("%d: %d\n", omp_get_thread_num (), i);

    return 0;
}
```

Output: thread no.: integer no.

```
sandhya@DESKTOP-3ERBGCG:~/HPC2024/OpenMP$ ./a.out

Enter value of max
5
0: 1
1: 2
2: 3
3: 4
4: 5
```

omp parallel for directive specifies that the for loop must be executed in parallel, i.e., its iterations must be divided among and executed by multiple threads running on all available processing units.

iterations of the parallel for loop are divided among threads where each iteration is executed by the thread it has been assigned to, and • once all iterations have been executed, all threads in the team are synchronized at the implicit barrier at the end of the parallel for loop and all slave threads are terminated. Finally, the execution proceeds sequentially and the master thread terminates the program by executing return 0.

This program does not specify how the iterations should be divided among threads (as explicit scheduling of iterations will be described later).

Lab Exercises:

- 1) Write a program in C to reverse the digits of the following integer array of size 9. Initialize the input array to the following values.

Input array: 18, 523, 301, 1234, 2, 14, 108, 150, 1928

Output array: 81, 325, 103, 4321, 2, 41, 801, 51, 8291

2) Write a program in C to simulate the all the operations of a calculator. Given inputs A and B, find the output for A+B, A-B, A*B and A/B.

3) Write a program in C to toggle the character of a given string.

Example: suppose the string is “HeLLo”, then the output should be “hElLO”.

4) Write a C program to read a word of length N and produce the pattern as shown in the example. Example: Input: PCBD Output: PCCBBBDDDDD

5) Write a C program to read two strings S1 and S2 of same length and produce the resultant string as shown below.

S1: string S2: length Resultant String: slternigtgh

6) Write a C program to perform Matrix times vector product operation.

7) Write a C program to read a matrix A of size 5x5. It produces a resultant matrix B of size 5x5. It sets all the principal diagonal elements of B matrix with 0. It replaces each row elements in the B matrix in the following manner. If the element is below the principal diagonal it replaces it with the maximum value of the row in the A matrix having the same row number of B. If the element is above the principal diagonal it replaces it with the minimum value of the row in the A matrix having the same row number of B.

Example:

A				
1	2	3	4	5
5	4	3	2	4
10	3	13	14	15
11	2	11	33	44
1	12	5	4	6

B				
0	1	1	1	1
5	0	2	2	2
15	15	0	3	3
44	44	44	0	2
12	12	12	12	0

- 8) Write a C program that reads a matrix of size MxN and produce an output matrix B of same size such that it replaces all the non-border elements of A with its equivalent 1's complement and remaining elements same as matrix A. Also produce a matrix D as shown below.

Example:

A				B				D			
1	2	3	4	1	2	3	4	1	2	3	4
6	5	8	3	6	10	111	3	6	2	7	3
2	4	10	1	2	11	101	1	2	3	5	1
9	1	2	5	9	1	2	5	9	1	2	5

- 9) Write a C program that reads a character type matrix and integer type matrix B of size MxN. It produces and output string STR such that, every character of A is repeated r times (where r is the integer value in matrix B which is having the same index as that of the character taken in A).

Example:

A				B			
p	C	a	P	1	2	4	3
e	X	a	M	2	4	3	2

Output string STR: pCCaaaaPPPeXXXXaaaMM

OpenMP programs on work-sharing constructs

Objectives:

In this lab, student will be able to

- ☐ **Understand Work-Sharing Concepts:** Explain the concept of work sharing in OpenMP and identify situations where it is beneficial.
- ☐ **Implement Parallel Loops:** Write OpenMP directives to parallelize loops using the parallel for construct to distribute loop iterations across multiple threads.

Lab Exercises:

1. Write a C program to:
 - a. Illustrates the fork-join pattern using OpenMP's parallel directive.
 - b. Illustrates the fork-join pattern using multiple OpenMP parallel directives and changing the number of threads two ways.
 - c. Illustrates the single-program-multiple-data (SPMD) pattern using two basic OpenMP commands.
2. Write a OpenMP program to calculate (i, x) for all the threads where i is an integer value and x is the thread_Id.
3. Write a OpenMP program that performs the sum of even numbers and odd numbers in a given input array. Create a separate thread to perform the sum of even numbers and odd numbers.
4. Write a OpenMP program to implement all the four basic operations of a calculator (Add, Sub, Mul, Div). Create a separate thread to perform the operations.
5. Write a OpenMP program for generating prime numbers from a given starting number to the given ending number.
6. Write a program in OpenMP to toggle the character of a given character array indexed by the thread_Id. Print the corresponding Thread_Id.
Example: suppose the string is "HeLLo", then the output should be "hElLO".
7. Write a program using OpenMP to compute the Fibonacci number for the following arrays of numbers: $A=\{10, 13, 5, 6\}$. Create a separate thread to perform the operations.

- a. For each matrix size, change the number of threads from 2,4,6 and 8 and plot the speedup versus the number of threads. Compute the efficiency.
8. Write an OpenMP program to perform Matrix times vector multiplication. Vary the matrix and vector size and analyze the speedup and efficiency of the parallelized code.
 9. Write an OpenMp program to read a matrix A of size 5x5. It produces a resultant matrix B of size 5x5. It sets all the principal diagonal elements of B matrix with 0. It replaces each row elements in the B matrix in the following manner. If the element is below the principal diagonal it replaces it with the maximum value of the row in the A matrix having the same row number of B. If the element is above the principal diagonal it replaces it with the minimum value of the row in the A matrix having the same row number of B. Analyze the speedup and efficiency of the parallelized code.
 10. Write a parallel program using OpenMP that reads a matrix of size MxN and produce an output matrix B of same size such that it replaces all the non-border elements of A with its equivalent 1's complement and remaining elements same as matrix A. Also produce a matrix D as shown below.

Example:

A

1	2	3	4
6	5	8	3
2	4	10	1
9	1	2	5

1	2	3	4
6	10	111	3
2	11	101	1
9	1	2	5

B

1	2	3	4
6	2	7	3
2	3	5	1
9	1	2	5

D

11. Write a parallel program in OpenMP to reverse the digits of the following integer array of size 9. Initialize the input array to the following values:
 - a. Input array: 18, 523, 301, 1234, 2, 14, 108, 150, 1928
 - b. Output array: 81, 325, 103, 4321, 2, 41, 801, 51, 8291

OpenMP programs on work-sharing constructs

Objectives:

In this lab, student will be able to

- Utilize Data Scoping: Apply appropriate data scoping clauses (private, shared, firstprivate, etc.) to manage data access and ensure correctness in parallelized loops.
- Optimize Loop Scheduling: Evaluate and choose between different loop scheduling strategies (static, dynamic, guided, etc.) to optimize load balancing and performance.
- Explore Nested Parallelism: Explore and implement nested parallel regions within loops to achieve finer-grained parallelism and optimize performance further.

Lab Exercises:

- 1) Write a parallel program using OpenMP to implement the Selection sort algorithm. Compute the efficiency and plot the speed up for varying input size and thread number.
- 2) Write a parallel program using OpenMP to implement sequential search algorithm. Compute the efficiency and plot the speed up for varying input size and thread number.
- 3) Write a parallel program using OpenMP to perform vector addition, subtraction, multiplication. Demonstrate task level parallelism. Analyze the speedup and efficiency of the parallelized code.
- 4) Write a parallel program using OpenMP to find sum of N numbers using the following constructs/clauses.
 - Critical section
 - Atomic
 - Reduction
 - Master
 - Locks
- 5) Write an OpenMP program to find the Summation of integers from a given interval. Analyze the performance of various iteration scheduling strategies.
- 6) Write a parallel program using OpenMP to compute π using random shooting.
Hint: Shoot randomly into a square $[0, 1] \times [0, 1]$ and count how many shots hit inside the unit circle and how many do not. Then calculate the ratio of number of points lied inside the circle and total number of generated points. We know that, area of the circle is πr^2 and area of the square is $4r^2$. Now, for a very large number of generated points,
No. of points generated inside the circle

$$\pi = 4 * \frac{\text{No. of points generated inside the circle}}{\text{Number of points generated inside the square}}$$

$$\pi = 4 * \frac{\text{No. of points generated inside the circle}}{\text{Number of points generated inside the square}}$$

Additional Exercises:

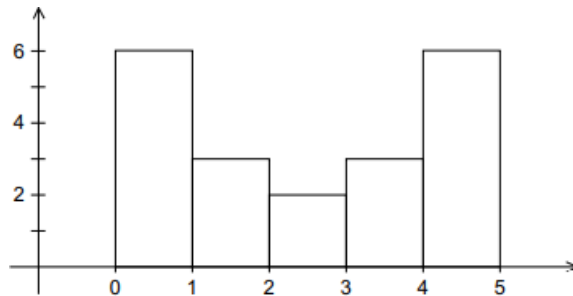
1) Write a parallel program using OpenMP to implement multi-threaded tokenizer for a text file. **Hint:** The tokens are just contiguous sequences of characters separated from the rest of the text by white space—spaces, tabs, or newlines. Assume that, the text file contains English text. A simple approach to this problem is to divide the input file into lines of text and assign the lines to the threads in a round-robin fashion. The first line goes to thread 0, the second goes to thread 1, . . . , the t^{th} goes to thread t , the $t + 1^{\text{st}}$ goes to thread 0, and so on. Each thread then tokenizes the input line and prints each of the tokens along with the ThreadID

2) Write a parallel program using OpenMP to generate the histogram of the given array A.

Hint: To generate histogram, we simply divide the range of the data up into equal sized sub intervals, or bins and determine the number of measurements (frequency) in each bin.

Example: suppose our data are

1.3, 2.9, 0.4, 0.3, 1.3, 4.4, 1.7, 0.4, 3.2, 0.3, 4.9, 2.4, 3.1, 4.4, 3.9, 0.4, 4.2, 4.5, 4.9, 0.9.



Where, Y axis represents the frequency of occurrence of the values and the x axis represents the bins.

3) Write a parallel program using OpenMP to find factorial of N numbers using the following constructs/clauses.

- Critical section
- Atomic
- Reduction
- Master
- Locks

Introduction to execution environment of MPI

Objectives:

In this lab, student will be able to

1. Understand the execution environment of MPI programs
2. Learn the various concept of parallel programming
3. Learn and use the Basics API available in MPI

I.Introduction

In order to reduce the execution time work is carried out in parallel. Two types of parallel programming are:

- Explicit parallel programming
- Implicit parallel programming

Explicit parallel programming – These are languages where the user has full control and has to explicitly provide all the details. Compiler effort is minimal.

Implicit parallel programming – These are sequential languages where the compiler has full responsibility for extracting the parallelism in the program.

Parallel Programming Models:

- Message Passing Programming
- Shared Memory Programming

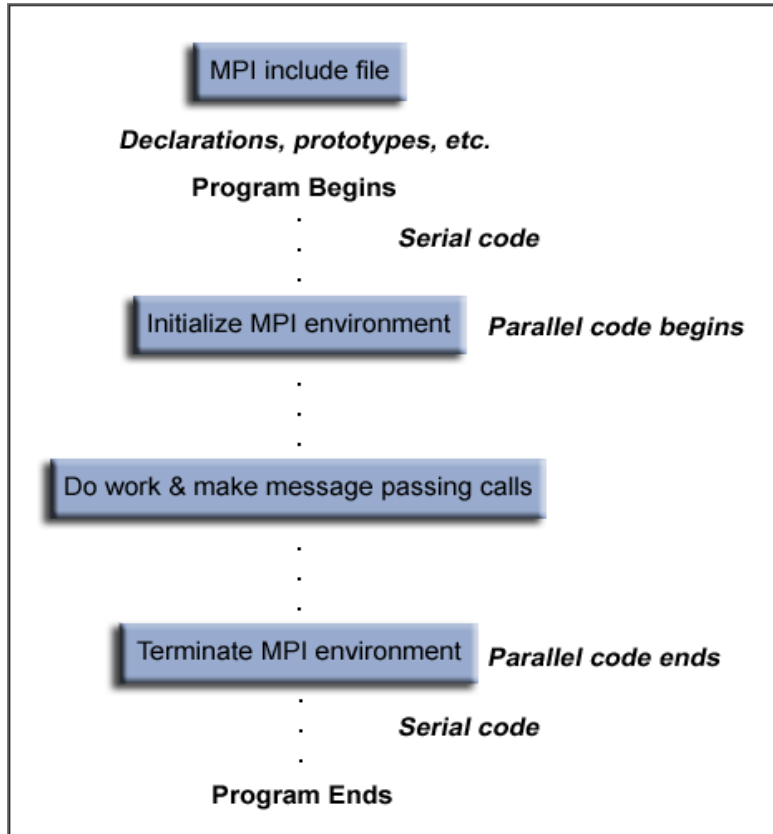
Message Passing Programming:

- In message passing programming, programmers view their programs (Applications) as a collection of co-operating processes with private (local) variables.
- The only way for an application to share data among processors is for programmer to explicitly code commands to move data from one processor to another.

Message Passing Libraries: There are two message passing libraries available. They are:

- PVM – Parallel Virtual Machine
- MPI – Message Passing Interface. It is a set of parallel APIs which can be used with languages such as C and FORTRAN.

II. MPI Program Structure:



Communicators and Groups:

- MPI assumes static processes.
- All the processes are created when the program is loaded.
- No process can be created or terminated in the middle of program execution.
- There is a default process group consisting of all such processes identified by **MPI_COMM_WORLD**.

III. MPI Environment Management Routines:

MPI_Init: Initializes the MPI execution environment. This function must be called in every MPI program, must be called before any other MPI functions and must be called only once in an MPI program.

```
MPI_Init (&argc,&argv);
```


MPI Comm size: Returns the total number of MPI processes to the variable size in the specified communicator, such as MPI_COMM_WORLD.

```
MPI_Comm_size(Comm,&size);
```

MPI Comm rank: Returns the rank of the calling MPI process within the specified communicator. Each process will be assigned a unique integer rank between 0 and size - 1 within the communicator MPI_COMM_WORLD. This rank is often referred to as a process ID.

```
MPI_Comm_rank Comm,&rank);
```

MPI Finalize: Terminates the MPI execution environment. This function should be the last MPI routine called in every MPI program. No other MPI routines may be called after it.

```
MPI_Finalize ();
```

Solved Example:

Write a program in MPI to print total number of process and rank of each process.

```
#include "mpi.h"
#include <stdio.h>
int main(int argc, char *argv[])
{
    int rank,size;

    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    printf("My rank is %d in total %d process",rank,size);
    MPI_Finalize();
    return 0;
}
```

Steps to execute a MPI program is provided in the form of video which is available in individual systems.

Lab Exercises:

1. Write a simple MPI program to find out pow (x, rank) for all the processes where 'x' is the integer constant and 'rank' is the rank of the process.
2. Write a program in MPI where even ranked process prints "Hello" and odd ranked process prints "World".

3. Write a program in MPI to simulate simple calculator. Perform each operation using different process in parallel.
4. Write a program in MPI to toggle the character of a given string indexed by the rank of the process. Hint: Suppose the string is HeLLLO and there are 5 processes, then process 0 toggle 'H' to 'h', process 1 toggle 'e' to 'E' and so on.

Additional Exercises:

1. Write a program in C to count the words in a file and sort it in descending order of frequency of words i.e. highest occurring word must come first and least occurring word must come last.
2. Write a MPI program to find the prime numbers between 1 and 100 using two processes.

Point to Point Communications in MPI

Objectives:

In this lab, student will be able to

1. Understand the different APIs used for point to point communication in MPI
2. Learn the different modes available in case of blocking send operation

Point to Point communication in MPI

- MPI point-to-point operations typically involve message passing between two, and only two, different MPI tasks. One task is performing a send operation and the other task is performing a matching receive operation.
- MPI provides both blocking and non-blocking send and receive operations.

Sending message in MPI

- **Blocked Send** sends a message to another processor and waits until the receiver has received it before continuing the process. Also called as **Synchronous send**.
- **Send** sends a message and continues without waiting. Also called as **Asynchronous send**.

There are multiple communication modes used in blocking send operation:

- **Standard mode**
- **Synchronous mode**
- **Buffered mode**

Standard mode

This mode blocks until the message is buffered.

```
MPI_Send(&Msg, Count, Datatype, Destination, Tag, Comm);
```

- First 3 parameters together constitute message buffer. The **Msg** could be any address in sender's address space. The **Count** indicates the number of data elements of a particular type to be sent. The **Datatype** specifies the message type. Some Data types available in MPI are: MPI_INT, MPI_FLOAT, MPI_CHAR, MPI_DOUBLE, MPI_LONG
- Next 3 parameters specify message envelope. The **Destination** specifies the rank of the process to which the message is to be sent.
- **Tag**: The **tag** is an integer used by the programmer to label different types of messages and to restrict message reception.

- **Communicator:** Major problem with tags is that they are specified by users who can make mistakes. **Context** are allocated at run time by the system in response to user request and are used for matching messages. The notions of **context and group** are combined in a single object called a communicator (**Comm**).
- The default process group is **MPI_COMM_WORLD**.

Synchronous mode

This mode requires a send to block until the corresponding receive has occurred.

```
MPI_Ssend(&Msg, Count, Datatype, Destination, Tag, Comm);
```

Buffered mode

```
MPI_Bsend(&Msg, Count, Datatype, Destination, Tag, Comm);
```

In this mode a send assumes availability of a certain amount of buffer space, which must be previously specified by the user program through a routine call that allocates a user buffer.

```
MPI-Buffer_attach(buffer, size);
```

This buffer can be released by

```
MPI-Buffer_detach(*buffer, *size);
```

Receiving message in MPI

```
MPI_Recv(&Msg, Count, Datatype, Source, Tag, Comm, &status);
```

- Receive a message and block until the requested data is available in the application buffer in the receiving task.
- The **Msg** could be any address in receiver's address space. The **Count** specifies number of data items. The **Datatype** specifies the message type. The **Source** specifies the rank of the process which has sent the message. The **Tag** and **Comm** should be same as that is used in corresponding send operation. The status is a structure of type status which contains following information: Sender's rank, Sender's tag and number of items received

Finding execution time in MPI

MPI_Wtime: Returns an elapsed wall clock time in seconds (double precision) on the calling processor.

- **MPI_Wtime ()**

Solved Example:

Write a MPI program using standard send. The sender process sends a number to the receiver. The second process receives the number and prints it.

```
#include "mpi.h"
#include <stdio.h>
int main(int argc, char *argv[])
{
    int rank,size,x;
    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Status status;
    if(rank==0)
    {
        Printf("Enter a value in master process:");
        scanf("%d",&x);
        MPI_Send(&x,1,MPI_INT,1,1,MPI_COMM_WORLD);
        fprintf(stdout,"I have send %d from process 0\n",x);
        fflush(stdout);
    }
    else
    {
        MPI_Recv(&x,1,MPI_INT,0,1,MPI_COMM_WORLD,&status);
        fprintf(stdout,"I have received %d in process 1\n",x);
        fflush(stdout);
    }
    MPI_Finalize();
    return 0;
}
```

Lab Exercises:

- 1) Write a MPI program using synchronous send. The sender process sends a word to the receiver. The second process receives the word, toggles each letter of the word and sends it back to the first process. Both process use synchronous send operations.
- 2) Write a MPI program where the master process (process 0) sends a number to each of the slaves and the slave processes receives the number and prints it. Use standard send.
- 3) Write a MPI program to read N elements of the array in the root process (process 0) where N is equal to the total number of process. The root process sends one value to each of the

slaves. Let even ranked process find square of the received element and odd ranked process find cube of received element. Use Buffered send.

- 4) Write a MPI program to read an integer value in the root process. Root process sends this value to process1, Process1 sends this value to Process2 and so on. Last process sends the value back to root process. When sending the value each process will first increment the received value by one. Write the program using point to point communication routines.
- 5) Write a MPI program to accept the input from .txt file by root process, and extract the text in each line. Send each statement to separate process and count the total number of word by each process. Display the word count by each process and send back the count to root process. Finally display the total word count by the root process.

Additional Exercises:

- 1) Write a MPI program to read N elements of an array in the root. Search a number in this array using root and another process. Print the result in the root.
- 2) Write a MPI program to read N elements of an array in the master process. Let N process including master process check the array values are prime or not.
- 3) Write a MPI program to read value of N in the root process. Using N processes including root find out $1! + (1+2) + 3! + (1+2+3+4) + 5! + (1+2+3+4+5+6) + \dots + n!$ or $(1+2+\dots+n)$ depending on whether n is odd or even and print the result in the root process.

Lab No 6:

Date:

Collective Communications and Error Handling in MPI

Objectives:

In this lab, student will be able to

1. Understand the usage of collective communication in MPI
2. Learn how to broadcast messages from root
3. Learn and use the APIs for distributing values from root and gathering the values in the root
4. Understand the different aggregate functions used in MPI
5. Learn how to write MPI programs using both point to point and collective communication routines
6. Learn and use the APIs for handling errors in MPI

Collective Communication routines

When **all processes** in a group participate in a global communication operation, the resulting communication is called a **collective communication**.

MPI_Bcast:

MPI_Bcast (Address, Count, Datatype, Root, Comm);

The process ranked **Root** sends the same message whose content is identified by the triple (Address,Count,Datatype) to all processes(including itself) in the communicator **Comm**.

MPI_Scatter:

MPI_Scatter(SendBuff, Sendcount, SendDatatype, RecvBuff, Recvcount, RecvDatatype, Root, Comm);

Ensures that the **Root** process sends out personalized messages, which are in rank order in its send buffer, to all the N processes (including itself).

MPI_Gather:

```
MPI_Gather( SendAddress, Sendcount, SendDatatype, RecvAddress, RecvCount,  
RecvDatatype, Root, Comm);
```

The **root** process receives a personalized message from all N processes. These N received messages are concatenated in rank order and stored in the receive buffer of the root process.

Total Exchange:

In routine **MPI_Alltoall()** each process sends a personalized message to every other process including itself. This operation is equivalent to N gathers, each by a different process and in all N^2 messages are exchanged.

Solved Example:

Write a MPI program to read N values of the array in the root process. Distribute these N values among N processes. Every process finds the square of the value it received. Let every process return these value to the root and root process gathers and prints the result. Use collective communication routines.

```
#include "mpi.h"
#include <stdio.h>

int main(int argc, char *argv[])
{
    int rank,size,N,A[10],B[10], c, i;

    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if(rank==0)
    {
        N=size;
        fprintf(stdout,"Enter %d values:\n",N);
        fflush(stdout);
        for(i=0; i<N; i++)
            scanf("%d",&A[i]);
    }
    MPI_Scatter(A,1,MPI_INT,&c,1,MPI_INT,0,MPI_COMM_WORLD);
    fprintf(stdout,"I have received %d in process %d\n",c,rank);
    fflush(stdout);

    c=c*c;
```



```

MPI_Gather(&c,1,MPI_INT,B,1,MPI_INT,0,MPI_COMM_WORLD);

if(rank==0)
{
    fprintf(stdout,"The Result gathered in the root \n");
    fflush(stdout);
    for(i=0; i<N; i++)
        fprintf(stdout,"%d \t",B[i]);
    fflush(stdout);
}

MPI_Finalize();
return 0;
}

```

I. Aggregation Functions

MPI provides two forms of aggregation

- **Reduction**
- **Scan**

Reduction:

MPI_Reduce (SendAddress, RecvAddress, Count, Datatype, Op, Root, Comm);

This routine reduces the partial values stored in **SendAddress** of each process into a final result and stores it in **RecvAddress** of the **Root** process. The reduction operator is specified by the **Op** field. Some of the reduction operator available in MPI are: MPI_SUM, MPI_MAX, MPI_MIN, MPI_PROD

Scan:

MPI_Scan (SendAddress, RecvAddress, Count, Datatype, Op, Comm);

This routine combines the partial values into N final results which it stores in the **RecvAddress** of the N processes. Note that root field is absent here. The scan operator is specified by the **Op** field. Some of the scan operator available in MPI are: MPI_SUM, MPI_MAX, MPI_MIN, MPI_PROD

MPI_Barrier(Comm) :This routine synchronizes all processes in the communicator **Comm**. They wait until all N processes execute their respective MPI_Barrier.

Note: All collective communication routines except MPI_Barrier, employ a standard blocking mode of point-to-point communication.

Error Handling in MPI:

- An MPI *communicator* is more than just a group of process that belong to it. Amongst the items that the communicator hides inside is an *error handler*. The error handler is called every time an MPI error is detected within the communicator.
- The predefined default error handler, which is called **MPI_ERRORS_ARE_FATAL**, for a newly created communicator or for MPI_COMM_WORLD is to *abort the whole parallel program* as soon as any MPI error is detected. There is another predefined error handler, which is called **MPI_ERRORS_RETURN**.
- The default error handler can be replaced with this one by calling function **MPI_Errhandler_set**, for example:

```
MPI_Errhandler_set(MPI_COMM_WORLD, MPI_ERRORS_RETURN);
```

- The only **error code** that MPI standard itself defines is **MPI_SUCCESS**, i.e., no error. But the meaning of an error code can be extracted by calling function **MPI_Error_string**. On top of the above MPI standard defines the so called **error classes**. The **error class** for a given error code can be obtained by calling function **MPI_Error_class**.
- Error classes can be converted to comprehensible error messages by calling the same function that does it for error codes, i.e., **MPI_Error_string**. The reason for this is that error classes are implemented as a subset of error codes.

Solved Example:

Write a MPI program using N processes to find $1! + 2! + \dots + N!$. Use collective communication routines.

```
#include <stdio.h>
#include "mpi.h"
int main(int argc, char* argv[])
{
    int rank, size, fact=1, factsum, i;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    for(i=1; i<=rank+1; i++)
        fact = fact * i;

    MPI_Reduce(&fact, &factsum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
```

```

    if(rank==0)
        printf("Sum of all the factorial=%d",factsum);

    MPI_Finalize();
    exit(0);
}

```

Lab Exercises:

- 1) Write a MPI program to read a value M and NxM elements in the root process. Root process sends M elements to each process. Each process finds average of M elements it received and sends these average values to root. Root collects all the values and finds the total average. Use collective communication routines. Use N number of processes.
- 2) Write a MPI Program to read two strings S1 and S2 of same length in the root process. Using N process including the root (string length is evenly divisible by N), produce the resultant string as shown below. Display the resultant string in the root process. Write the program using Collective communication routines.

Example:

String S1: string String S2: length Resultant String : slternigtgh

- 3) Write a MPI program using N processes to find $1! + 2! + \dots + N!$. Use scan.
- 4) Write a MPI program to read a 3 x 3 matrix. Enter an element to be searched in the root process. Find the number of occurrences of this element in the matrix using three processes.
- 5) Write a MPI program to handle different errors using error handling routines.
- 6) Write a MPI program to read 4 x 4 matrix display the following output using four processes

I/p matrix: 1	2	3	4	O/p matrix: 1	2	3	4
1	2	3	1	2	4	6	5
1	1	1	1	3	5	7	6
2	1	2	1	5	6	9	7

Additional Exercises:

- 1) Write a MPI Program to read a string of length M in the root process. Using N processes (N evenly divides M) including the root toggle the characters and find the ASCII values of

these toggled characters. Display the toggled characters and ASCII values in the root process.

- 2) Write a program to read a value M and NxM number of elements in the root. Using N processes do the following task. Find the square of first M numbers, Find the cube of next M numbers and so on. Print the results in the root.
- 3) Write a program to read a value M and NxM number of elements in the root. Using N number of processes find the sum $1+2+...+array$ element of each element and print the result in the root.

I/p: M=2 N=3

Array: 2 4 3 2 5 3

Result: 3 10 6 3 15 6

- 4) Write a MPI program to read a word of length N. Using N processes including the root get output word with the pattern as shown in example. Display the resultant output word in the root.

Example: Input : PCAP Output : PCCAAAPPPP

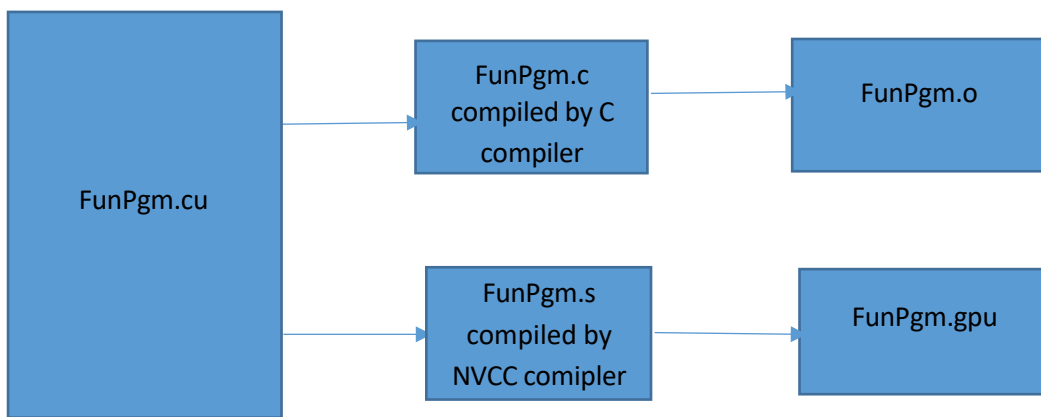
Programs on arrays in CUDA

Objectives:

In this lab, student will be able to

1. Know the basics of Computing Unified Device Architecture (CUDA).
2. Learn program structure of CUDA.
3. Learn about CUDA 1D blocks and threads
4. Write simple programs on one dimensional arrays
5. Learn mathematical functions in CUDA

About CUDA: CUDA is a platform for performing massively parallel computations on graphics accelerators. CUDA was developed by NVIDIA. It was first available with their G8X line of graphics cards. CUDA presents a unique opportunity to develop widely-deployed parallel applications. The CUDA programs are compiled as follows.



FunPgm.cu is compiled by both C compiler and Nvidia CUDA C compiler (NVCC compiler). If you have both main.c and Funpgm.cu then you can call cuda API's in main.c but keep in mind that you cannot call kernel from main.c. To call the kernel file extension must be .cu.

As in CUDA, CPU is the host and its memory the host memory and GPU is the device and its memory device memory. Serial code will be run on host and parallel code will be run on device.

1. Copy data from host memory to device memory.
2. Load device program and execute, caching data on chip for performance.
3. Copy result from device memory to host memory.

CUDA threads, blocks and grid

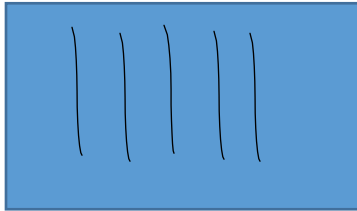
Thread – Distributed by the CUDA runtime. A single path of execution there can be multiple threads in a program.

(identified by threadIdx)

CUDA Thread

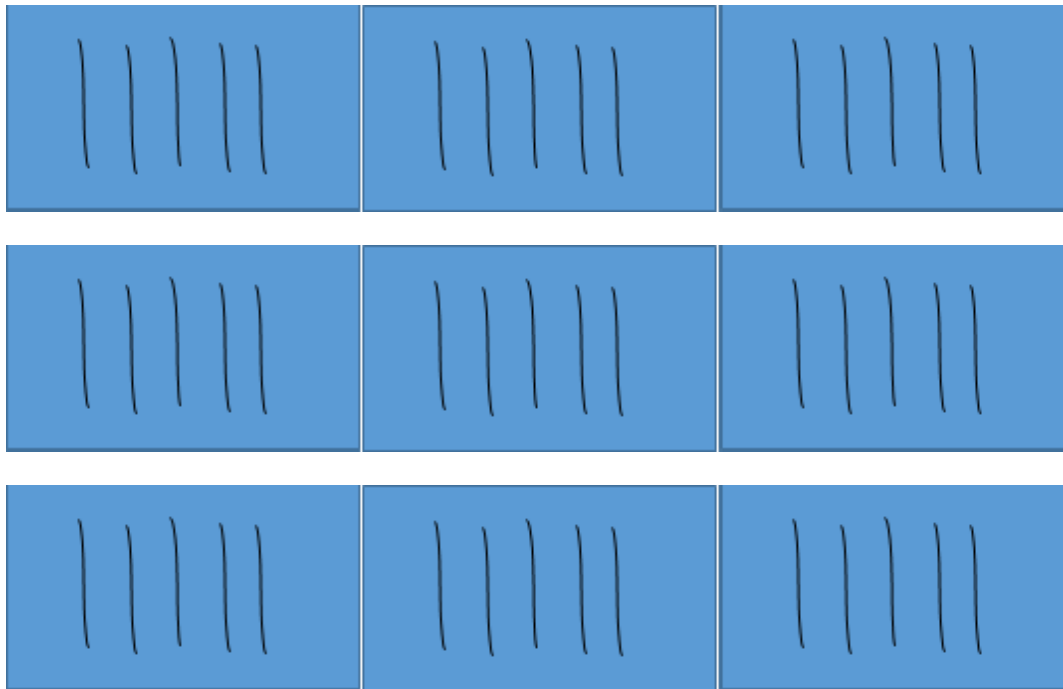
Block – A user defined group of 1 to 512 threads.

(identified by blockIdx)



CUDA Block

Grid – A group of one or more blocks. A grid is created for each CUDA kernel function



CUDA
GRID

```
}  
1D grid of 2D blocks  
__device__  
int getGlobalIdx_1D_2D(){  
    return blockIdx.x * blockDim.x * blockDim.y  
    + threadIdx.y * blockDim.x + threadIdx.x;  
}
```

```

}
1D grid of 3D blocks
__device__
int getGlobalIdx_1D_3D(){
return blockIdx.x * blockDim.x * blockDim.y * blockDim.z
+ threadIdx.z * blockDim.y * blockDim.x
+ threadIdx.y * blockDim.x + threadIdx.x;
}
2D grid of 1D blocks
__device__ int getGlobalIdx_2D_1D(){
int blockIdx = blockIdx.y * gridDim.x + blockIdx.x;
int threadId = blockIdx * blockDim.x + threadIdx.x;
return threadId;
}
2D grid of 2D blocks
__device__
int getGlobalIdx_2D_2D(){
int blockIdx = blockIdx.x + blockIdx.y * gridDim.x;
int threadId = blockIdx * (blockDim.x * blockDim.y)
+ (threadIdx.y * blockDim.x) + threadIdx.x;
return threadId;
}
2D grid of 3D blocks
__device__
int getGlobalIdx_2D_3D(){
int blockIdx = blockIdx.x + blockIdx.y * gridDim.x;
int threadId = blockIdx * (blockDim.x * blockDim.y * blockDim.z)
+ (threadIdx.z * (blockDim.x * blockDim.y))
+ (threadIdx.y * blockDim.x) + threadIdx.x;
return threadId;
}
3D grid of 1D blocks
__device__
int getGlobalIdx_3D_1D(){
int blockIdx = blockIdx.x + blockIdx.y * gridDim.x
+ gridDim.x * gridDim.y * blockIdx.z;
int threadId = blockIdx * blockDim.x + threadIdx.x;
return threadId;
}
3D grid of 2D blocks
__device__
int getGlobalIdx_3D_2D(){
int blockIdx = blockIdx.x + blockIdx.y * gridDim.x
+ gridDim.x * gridDim.y * blockIdx.z;
int threadId = blockIdx * (blockDim.x * blockDim.y)
+ (threadIdx.y * blockDim.x) + threadIdx.x;

```

```

return threadIdx;
}
3D grid of 3D blocks
__device__
int getGlobalIdx_3D_3D(){
int blockIdx = blockIdx.x + blockIdx.y * gridDim.x
+ gridDim.x * gridDim.y * blockIdx.z;
int threadIdx = blockIdx * (blockDim.x * blockDim.y * blockDim.z)
+ (threadIdx.z * (blockDim.x * blockDim.y))
+ (threadIdx.y * blockDim.x) + threadIdx.x;
return threadIdx;
}

```

Solved Exercise: Program to add two numbers.

```

__global__ void add(int *a, int *b, int *c) {
    *c = *a + *b;
}

int main(void) {
    int a, b, c;           // host copies of variables a, b & c
    int *d_a, *d_b, *d_c; // device copies of variables a, b & c
    int size = sizeof(int);

    // Allocate space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);
    // Setup input values
    a = 3;
    b = 5;
    // Copy inputs to device
    cudaMemcpy(d_a, &a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, &b, size, cudaMemcpyHostToDevice);
    // Launch add() kernel on GPU
    add<<<1,1>>>>(d_a, d_b, d_c);
    // Copy result back to host
    cudaMemcpy(&c, d_c, size, cudaMemcpyDeviceToHost);
    printf("Result : %d",c);
    // Cleanup
    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
    return 0;
}

```


Explanation:

add is the function which runs on device.

```
cudaMalloc ((void **)&d_a, size);
```

cudaMalloc will allocate memory of size bytes given as second argument to variable passed as first argument.

```
cudaMemcpy (Destination,Source,Size,Direction);  
cudaMemcpy (d_a, &a, size, cudaMemcpyHostToDevice);  
cudaMemcpy (&c, d_c, size, cudaMemcpyDeviceToHost);
```

cudaMemcpy copies the variables from host to device or device to host based on the direction which is either cudaMemcpyHostToDevice or cudaMemcpyDeviceToHost. Value of size bytes long is copied from source to destination.

cudaFree frees the memory allocated by cudaMalloc.

The add function is called like this add<<<1,1>>>(d_a,d_b,d_c). The add is followed by three angular brackets then the number of blocks, threads per block then corresponding closing angular brackets then how many arguments the function add takes is enclosed within parenthesis. If you want to add N elements you can achieve it in two ways either having N blocks or having N threads.

That is pass an array with following function calls

add<<< N,1>>> (d_a,d_b,d_c) or add<<< 1, N>>> (d_a,d_b,d_c)

Few Mathematical functions in CUDA:

Major Single-Precision floating point functions: Single precision functions work on float value(32 bit). A float value is stored in IEEE 754 format.

Function	Description
sqrtf(x)	Square root function
expf(x)	Exponentiation function. Base = e
exp2f(x)	Exponentiation function. Base = 2
exp10f(x)	Exponentiation function. Base = 10
logf(x)	Logarithmic function. Base=e
log2f(x)	Logarithmic function. Base=2
log10f(x)	Logarithmic function. Base=10

sinf(x)	sine function
cosf(x)	cos function
tanf(x)	tan function
powf(x,y)	power function
truncf(x)	truncation function
roundf(x)	round function
ceilf(x)	ceil function
floorf(x)	floor function

Major Double-Precision floating point functions: Double precision functions work on double value(64 bit). A double value is stored in IEEE 754 format.

Function	Description
sqrt(x)	Square root function
exp(x)	Exponentiation function. Base = e
exp2(x)	Exponentiation function. Base = 2
exp10(x)	Exponentiation function. Base = 10
log(x)	Logarithmic function. Base=e
log2(x)	Logarithmic function. Base=2
log10(x)	Logarithmic function. Base=10
sin(x)	sine function
cos(x)	cos function
tan(x)	tan function
pow(x,y)	power function
trunc(x)	truncation function
round(x)	round function
ceil(x)	ceil function
floor(x)	floor function

Steps to execute a CUDA program is provided in the form of video which is made available in individual systems.

Lab Exercises:

1. Write a program in CUDA to add two vectors using
 - a) block size as N
 - b) N threads
 - c) Varying block size to handle N elements with thread size constant as 256
2. Write a program in CUDA which performs convolution operation on the input N elements using a mask array with M elements to produce the resultant N elements.
3. Write a program in CUDA to process a 1D array containing angles in radians to generate sine of the angles in the output array. Use appropriate function.

Additional Exercises:

1. Write a program in CUDA to sort every row of a matrix using selection sort.
2. Write a program in CUDA to perform odd even transposition sort in parallel.

Programs on strings in CUDA

Objectives:

In this lab, student will be able to

1. Write simple programs on Strings
2. Learn to compute time of kernel execution
3. Learn about atomic functions
4. Learn to handle errors in the kernel

Arithmetic functions:

In a multithreaded scenario, the issue of data inconsistency will arise, if multiple threads modify a single shared memory variable. To overcome this atomic functions need to be used. List of atomic functions, their syntax and explanation is provided below.

atomicAdd():

```
int atomicAdd (int* address, int val);  
unsigned int atomicAdd(unsigned int* address, unsigned int val);  
float atomicAdd(float* address, float val);  
double atomicAdd(double* address, double val);
```

Reads the 16-bit, 32-bit or 64-bit word old located at the address address in global or shared memory, computes (old + val), and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns old.

atomicSub():

```
int atomicSub(int* address, int val);  
unsigned int atomicSub(unsigned int* address, unsigned int val);
```

Reads the 32-bit word old located at the address address in global or shared memory, computes (old - val), and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns old.

atomicExch():

```
int atomicExch(int* address, int val);
```

```
unsigned int atomicExch(unsigned int* address, unsigned int val);
```

```
float atomicExch(float* address, float val);
```

Reads the 32-bit word old located at the address address in global or shared memory and stores val back to memory at the same address. These two operations are performed in one atomic transaction. The function returns old.

atomicMin():

```
int atomicMin(int* address, int val);
```

```
unsigned int atomicMin(unsigned int* address, unsigned int val);
```

Reads the 32-bit word old located at the address address in global or shared memory, computes the minimum of old and val, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns old.

atomicMax():

```
int atomicMax(int* address, int val);
```

```
unsigned int atomicMax(unsigned int* address, unsigned int val);
```

Reads the 32-bit word old located at the address address in global or shared memory, computes the maximum of old and val, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns old.

atomicInc():

```
unsigned int atomicInc(unsigned int* address, unsigned int val);
```

Reads the 32-bit word old located at the address address in global or shared memory, computes $((old \geq val) ? 0 : (old+1))$, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns old.

atomicDec():

```
unsigned int atomicDec(unsigned int* address, unsigned int val);
```

Reads the 32-bit word old located at the address address in global or shared memory, computes $((old == 0) \vee (old > val)) ? val : (old - 1)$, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns old.

Solved Example:

A CUDA program which takes a string as input and determines the number of occurrences of a character 'a' in the string. This program uses atomicAdd() function.

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <conio.h>
#define N 1024

__global__ void CUDACount(char* A, unsigned int *d_count){
    int i = threadIdx.x;
    if(A[i]=='a')
        atomicAdd(d_count,1);
}

int main(){

    char A[N];
    char *d_A;
    unsigned int *count=0,*d_count,*result;
    printf("Enter a string");
    gets(A);
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    cudaEventRecord(start, 0);
    cudaMalloc((void**)&d_A, strlen(A)*sizeof(char));
    cudaMalloc((void **)&d_count,sizeof(unsigned int));
    cudaMemcpy(d_A, A, strlen(A)*sizeof(char), cudaMemcpyHostToDevice);
    cudaMemcpy(d_count,count,sizeof(unsigned int),cudaMemcpyHostToDevice);
    cudaError_t error =cudaGetLastError();
    if (error != cudaSuccess)
    {
        printf("CUDA Error1: %s\n", cudaGetErrorString(error));
    }

    CUDACount<<<1,strlen(A)>>>(d_A,d_count);
```

```

error =cudaGetLastError();
if (error != cudaSuccess)
    {
        printf("CUDA Error2: %s\n", cudaGetErrorString(error));
    }

cudaEventRecord(stop, 0);
cudaEventSynchronize(stop);
float elapsedTime;
cudaEventElapsedTime(&elapsedTime, start, stop);
cudaMemcpy(result, d_count, sizeof(unsigned int), cudaMemcpyDeviceToHost);
printf("Total occurrences of a=%u",result);
printf("Time Taken=%f",elapsedTime);
cudaFree(d_A);
cudaFree(d_count);
printf("\n");
getch();
return 0;
}

```

Explanation:

The kernel uses atomicAdd function with 1 as the value each time a character 'a' occurs in the string. The instructions given in bold are present to find the time. As in CUDA you need to declare an event, register the event and record the time before kernel execution and after kernel execution. You have to synchronize the event so that main thread can capture the time of execution of kernel. After that **cudaEventElapsedTime(&elapsedTime, start, stop)** will give the difference between the recorded stop and start time and store the value in the variable elapsedTime which is of type float. A negative time value means there is something wrong in the CUDA code. To find it out you use the code given in bold and italics. It will display the error message present in CUDA code. Call it once before calling the kernel and once after calling kernel. If first call throws an error message then error is present in CUDA API which precedes the kernel. If second call throws the error message then error is present in the kernel code. .

Lab Exercises:

- 1) Write a CUDA program that takes a string S as input and one integer value N. Produces output string N times as follows in parallel:
I/p: S = Hello N = 3
O/p String: HelloHelloHello (Each thread copies entire S)
- 2) Write a CUDA program which reads a string consisting of N words and reverse each word of it in parallel.

- 3) Write a program in CUDA to count the number of times a given word is repeated in a sentence. (Use Atomic function)
- 4) Write a CUDA program that reads a string with N words and reverse entire string in parallel.

Additional Exercises:

1) Write an CUDA program that takes a string S as input and one integer value N. Produces string N times as follows in parallel:

I/p: S = Hello N = 3

O/p String: HelloHelloHello (Every thread copies same character from the Input N times to the required position)

2) Write a CUDA program which reads a string S and produces output string T as follows:

S: Hai T: Haaiii

(Every thread stores a character from input string S required number of times in T)

Programs on Matrix using CUDA

Objectives:

In this lab, student will be able to

1. Understand how to write kernel code in CUDA to perform operations on matrix
2. Learn about CUDA 2D blocks/threads
3. Write simple programs on two dimensional arrays

Solved Exercise:

Write a program in CUDA to find transpose of a matrix in parallel.

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>

__global__ void transpose(int *a, int *t)
{
    int n=threadIdx.x, m=blockIdx.x, size=blockDim.x, size1=gridDim.x;
    t[n*size1+m]=a[m*size+n];
}

int main(void)
{
    int *a,*t, m,n,i,j;
    int *d_a,*d_t;
    printf("Enter the value of m: ");scanf("%d",&m);
    printf("Enter the value of n: ");scanf("%d",&n);
    int size=sizeof(int)*m*n;
    a=(int*)malloc(m*n*sizeof(int));
    t=(int*)malloc(m*n*sizeof(int));
    printf("Enter input matrix:\n");
    for(i=0;i<m*n;i++)
        scanf("%d",&a[i]);

    cudaMalloc((void**)&d_a,size);
    cudaMalloc((void**)&d_t,size);

    cudaMemcpy(d_a,a,size,cudaMemcpyHostToDevice);

    transpose<<<m,n>>>>(d_a,d_t);
```

```

    cudaMemcpy(t,d_t,size,cudaMemcpyDeviceToHost);
    printf("Result vector is:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<m;j++)
            printf("%d\t",t[i*m+j]);
        printf("\n");
    }

    getchar();
    cudaFree(d_a);
    cudaFree(d_t);
    return 0;

}

```

Lab Exercises:

1. Write a program in CUDA to read MxN matrix. Replace 1st row of this matrix by same elements, 2nd row elements by square of each element and 3rd row elements by cube of each element and so on.
2. Write a program in CUDA to add two Matrices using
 - a. Each row of resultant matrix to be computed by one thread.
 - b. Each column of resultant matrix to be computed by one thread.
 - c. Each element of resultant matrix to be computed by one thread.
3. Repeat the above exercise for matrix multiplication.
4. Write an CUDA program that reads a matrix A of size M*N and produces a output matrix B of same size such that it replaces all the non-border elements(numbers in bold) of A with its equivalent 1's complement and remaining elements same as matrix A.

A				B			
1	2	3	4	1	2	3	4
6	5	8	3	6	10	111	3
2	4	10	1	2	11	101	1
9	1	2	5	9	1	2	5

Additional Exercises:

- 1) Write an CUDA program which reads an input matrix A of size MxN. It produces an output matrix B of size MxN such that, each element of the output matrix is calculated in parallel. Each element in the output matrix is a total sum of row sum and column sum of those elements that lies in the same row and same column index of that element in the input matrix.

Example: A B

4	2	3
4	5	6

O/p: 11	13	15
20	22	24

- 2) Write an CUDA program that reads a MxN matrix A and produces a resultant matrix B of same size as follows: Replace all the even numbered matrix elements with their row sum and odd numbered matrix elements with their column sum.
- 3) Write an CUDA program to read a matrix A of size NxN. It replaces the principal diagonal elements with zero. Elements above the principal diagonal by their factorial and elements below the principal diagonal by their sum of digits.
- 4) Write a program in CUDA to improve the performance of above Sparse Matrix - Vector multiplication.

Programs on CUDA Device memory types and synchronization

Synchronization: CUDA allows threads in the same block to coordinate their activities using a barrier synchronization function, `_syncthreads()`. When a kernel function calls `_syncthreads()`, the thread that executes the function call will be held at the calling location until every thread in the block reaches the location. This ensures that all threads in a block have completed a phase of their execution of the kernel before any moves on to the next phase.

Shared Variables: Accessing shared memory is extremely fast and highly parallel. If a variable declaration is preceded by the keyword `__shared__`, it declares a shared variable in CUDA. Such declarations typically reside within a kernel function or a device function. The scope of a shared variable is within a thread block means all threads in a block see the same version of a shared variable. The lifetime of a shared variable is within the duration of the kernel. Shared variables are an efficient means for threads within a block to collaborate with each other. Shared memory is fast but it is small. A common strategy is partition the data into subsets called **tiles** so that each tile fits into the shared memory.

Constant Variables: If a variable declaration is preceded by the keyword `__constant__`, it declares a constant variable in CUDA. Declaration of constant variables must be outside any function body. The scope of a constant variable is all grids, meaning that all threads in all grids see the same version of a constant variable. The lifetime of a constant variable is the entire application execution. Constant variables are stored in the global memory but are cached for efficient access. With appropriate access patterns, accessing constant memory is extremely fast and parallel. Currently, the total size of constant variables in an application is limited at 65,536 bytes. One may need to break up the input data volume to fit within this limitation.

Device Variables: A variable whose declaration is preceded only by the keyword `__device__` is a global variable and will be placed in global memory. Accesses to a global variable are slow. However, global variables are visible to all threads of all kernels. Their contents also persist through the entire execution. Thus, global variables can be used as a means for threads to collaborate across blocks. Global variables are often used to pass information from one kernel invocation to another kernel invocation.

Solved Exercise:

Write a program in CUDA to perform tiled matrix multiplication using 2D Grid and 2D Block

```
//Matrix multiplication of 4x4 matrix
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <cuda_runtime.h>
```

```

#define BLOCK_WIDTH 2
#define TILE_WIDTH 2
#define WIDTH 4

__global__ void MatMulElementThreadShared(int *a, int *b, int *c) {
    __shared__ int MDs[TILE_WIDTH][TILE_WIDTH];
    __shared__ int NDs[TILE_WIDTH][TILE_WIDTH];
    int m;
    int bx=blockIdx.x; int by=blockIdx.y;
    int tx=threadIdx.x; int ty=threadIdx.y;

    int Row=by*TILE_WIDTH + ty;
    int Col= bx*TILE_WIDTH  + tx;

    int Pvalue=0;
    for(m=0; m<WIDTH/TILE_WIDTH; m++)
    {
        MDs[ty][tx]=a[Row*WIDTH+m*TILE_WIDTH+tx];
        NDs[ty][tx]=b[(m*TILE_WIDTH+ty)*WIDTH+Col];

        __syncthreads();

        for (int k = 0; k < TILE_WIDTH; k++)
        {
            Pvalue += MDs[ty][k]*NDs[k][tx];
        }
        __syncthreads();
    }
    c[Row*WIDTH+Col] = Pvalue;
}

int main() {
    int *matA, *matB, *matProd;
    int *da, *db, *dc;

    printf("\n== Enter elements of Matrix A (4x4) ==\n");

    matA = (int*)malloc(sizeof(int) * WIDTH * WIDTH);
    for(int i = 0; i < WIDTH * WIDTH; i++)
    {
        scanf("%d", &matA[i]);
    }

    printf("\n== Enter elements of Matrix B (4x4) ==\n");

```

```

matB = (int*)malloc(sizeof(int) * WIDTH * WIDTH);
for(int i = 0; i < WIDTH * WIDTH; i++)
{
    scanf("%d", &matB[i]);
}
matProd = (int*)malloc(sizeof(int) * WIDTH * WIDTH);

cudaMalloc((void **) &da, sizeof(int) * WIDTH * WIDTH);
cudaMalloc((void **) &db, sizeof(int) * WIDTH * WIDTH);
cudaMalloc((void **) &dc, sizeof(int) * WIDTH * WIDTH);

cudaMemcpy(da, matA, sizeof(int) * WIDTH * WIDTH, cudaMemcpyHostToDevice);
cudaMemcpy(db, matB, sizeof(int) * WIDTH * WIDTH, cudaMemcpyHostToDevice);
int NumBlocks = WIDTH / BLOCK_WIDTH;
dim3 grid_conf (NumBlocks, NumBlocks);
dim3 block_conf (BLOCK_WIDTH, BLOCK_WIDTH);

MatMulElementThreadShared<<<grid_conf, block_conf>>>(da, db, dc);

cudaMemcpy(matProd, dc, sizeof(int) * WIDTH * WIDTH, cudaMemcpyDeviceToHost);
printf("\n-=Result of Addition=-\n");
printf("-----\n");
for (int i = 0; i < m; i++ ) {
    for (int j = 0; j < n; j++) {
        printf("%6d ", matProd[i * n + j]);
    }
    printf("\n");
}
cudaFree(da);
cudaFree(db);
cudaFree(dc);
free(matA);
free(matB);
free(matProd);
return 0;
}

```

Lab Exercises:

1. Write a program in CUDA to improve performance of 1D parallel convolution using constant Memory.
2. Write a program in CUDA which performs tiled 1D convolution operation on the input N elements using a mask array with M elements to produce the resultant N elements.
3. Write a program in CUDA to perform inclusive scan algorithm.

Additional Exercise:

1. Write a program in CUDA which displays a shopping mall item menu with its price. N number of friends are allowed to purchase as many items they want. Calculate the total purchase done by N friends.

REFERENCES

- 1) Georg Hager Gerhard Wellein, Introduction to High Performance Computing for Scientists and Engineers, 1st Edition, CRC Press, 2010.
- 2) Barbara Chapman, Gabriele Jost, Ruud van der Pas, Using OpenMP Portable Shared Memory Parallel Programming MIT Press, 2007.
- 3) David B. Kirk and Wen-mei W. Hwu. Programming Massively Parallel Processors A Hands-on Approach, 4th Edition - May 28, 2022, Elsevier.
- 4) S. Cook, “CUDA Programming: A Developer's Guide to Parallel Computing with GPUs”, 1st edition, Morgan Kaufmann, 2012.
- 5) J. Sanders, E. Kandrot, “CUDA by example: an introduction to general-purpose GPU programming”, Addison-Wesley Professional, 2010.